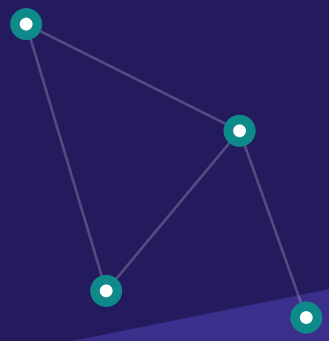
LANGGRAPH MASTERY SERIES



PHASE 2

Core LangGraph

Build working agents with LangGraph's primitives.
From StateGraph to checkpoints, HITL, and streaming.

WHAT YOU WILL MASTER

- StateGraph: nodes, edges, entrypoint
- Typed state with TypedDict and Pydantic + reducers
- Conditional edges and the Command API
- ToolNode and bind_tools: the tool-calling loop
- Checkpointers: MemorySaver, SQLite, Postgres
- Human-in-the-loop with interrupt()
- Streaming: values, updates, messages, custom

Three hands-on projects: a from-scratch ReAct agent, a memory chatbot, an approval agent.

Built on the official LangGraph documentation (docs.langchain.com) · LangGraph v1.1, 2026

From the loop in your head to a graph in code

In Phase 1 you built a ReAct agent as a hand-written Python loop. LangGraph is the framework that turns that loop into a durable, streamable, inspectable **graph**. This phase introduces every core primitive you need to build real agents, then puts them to work in three projects: a ReAct agent built from scratch, a chatbot that remembers across turns, and a tool-calling agent that pauses for human approval.

Install it with a single command. LangGraph requires Python 3.10+ and is published on PyPI; this guide targets the v1.x line (the current release at the time of writing is v1.1).

```
pip install -U langgraph

# Plus a model provider, e.g. Anthropic or OpenAI:
pip install -U "langchain[anthropic]"
```

> **LangGraph vs. LangChain**
> LangGraph is the low-level orchestration runtime: durable execution, streaming, human-in-the-loop, and persistence. LangChain provides the model and tool integrations plus a prebuilt agent. You can use LangGraph without LangChain, but we use LangChain components for models and tools throughout.

1. StateGraph: nodes, edges, entrypoint

A LangGraph application is a **StateGraph**: a set of **nodes** (steps) connected by **edges** (transitions), all operating on a shared **state**. You build the graph, then **compile** it into a runnable. The smallest possible graph has one node:

The LangGraph 'hello world'. START and END are sentinel nodes marking entry and exit.

```
from langgraph.graph import StateGraph, MessagesState, START, END

def mock_llm(state: MessagesState):
    return {"messages": [{"role": "ai", "content": "hello world"}]}

graph = StateGraph(MessagesState)
graph.add_node(mock_llm)            # a node is just a function of state
graph.add_edge(START, "mock_llm")   # START -> node
graph.add_edge("mock_llm", END)     # node -> END
graph = graph.compile()             # produce a runnable

graph.invoke({"messages": [{"role": "user", "content": "hi!"}]})
```

- **Nodes** are Python functions whose first argument is the current state. They return a *partial update* to the state — never mutate state in place.
- **Edges** wire nodes together. A normal edge `add_edge("a", "b")` always goes a→b. `START` marks the entrypoint; reaching `END` (or having no next node) halts the graph.
- **Entrypoint:** set it with `add_edge(START, "node")` or `builder.set_entry_point("node")`.
- **compile()** validates the structure (e.g. catches orphaned nodes) and returns the runnable you call with `invoke`, `stream`, etc.

> **The execution model: super-steps**
> LangGraph executes in discrete 'super-steps'. Each super-step runs all nodes scheduled for that tick (possibly in parallel), then applies their updates to state. This is the unit at which checkpoints are saved and the recursion limit is counted.

2. Typed state: TypedDict, Pydantic, and reducers

State is the single source of truth threaded through every node. Its schema can be a `TypedDict`, a `dataclass`, or a Pydantic `BaseModel`. `TypedDict` is the common default; Pydantic adds runtime validation of inputs.

A TypedDict state schema and a node that returns a partial update.

```
from typing_extensions import TypedDict
from langchain.messages import AnyMessage

class State(TypedDict):
    messages: list[AnyMessage]
    extra_field: int

def node(state: State):
    # read from state, return a PARTIAL update (only the keys you change)
    return {"extra_field": 10}
```

Reducers: how updates are combined

By default, a node's update **overwrites** the corresponding state key. But for some keys — above all the message history — you want to **append** instead. A **reducer** is a function that controls how an update is merged into the existing value. You attach one by annotating the field. The most important reducer is `add_messages`, which appends new messages and intelligently handles updates to existing ones.

Reducers via Annotated. MessagesState is a prebuilt shortcut for the common case.

```
from typing_extensions import Annotated, TypedDict
from langchain.messages import AnyMessage
from langgraph.graph.message import add_messages

class State(TypedDict):
    # updates to `messages` are APPENDED, not overwritten
    messages: Annotated[list[AnyMessage], add_messages]
    extra_field: int

# LangGraph ships a prebuilt MessagesState with exactly this messages field:
from langgraph.graph import MessagesState
class State(MessagesState):       # inherit the messages+add_messages field
    extra_field: int
```

Pattern	Behaviour	When to use
No reducer (default)	New value overwrites the old	Scalars and fields you fully replace each step
`add_messages`	Append messages; update by id	The conversation history
`operator.add`	Concatenate lists / sum numbers	Accumulating results from parallel nodes
`Overwrite(...)`	Bypass a reducer for one update	Resetting an accumulated field on purpose

> **Pydantic state — know the limits**
>
> Pydantic models validate inputs to the first node, which is great for catching bad input early. But the graph output is a plain dict (not a model instance), validation runs only on the first node's input, and recursive validation can be slow. For performance-sensitive graphs a dataclass is often a better middle ground.

Runtime configuration and context

Sometimes you want to configure a run without polluting the state — for example, which model to use or which user is calling. Pass a typed `context` and read it in nodes via the injected `Runtime` object.

Runtime context keeps per-run configuration out of the state.

```
from dataclasses import dataclass
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.runtime import Runtime

@dataclass
class Context:
    model_provider: str = "anthropic"

def call_model(state: MessagesState, runtime: Runtime[Context]):
    provider = runtime.context.model_provider   # read runtime config
    ...

builder = StateGraph(MessagesState, context_schema=Context)
# ...
graph = builder.compile()
graph.invoke({"messages": [...]}, context={"model_provider": "openai"})
```

3. Conditional edges and the Command API

A graph that only moves in straight lines is just a chain. **Conditional edges** are what give agents their branching and looping. A conditional edge runs a small routing function over the current state and returns the name of the next node (or `END`).

A conditional edge creates the branch; the b->a edge creates the loop. This is a ReAct skeleton.

```
from typing import Literal
from langgraph.graph import StateGraph, START, END

def route(state: State) -> Literal["b", "__end__"]:
    if termination_condition(state):
        return END
    return "b"

builder = StateGraph(State)
builder.add_node(a); builder.add_node(b)
builder.add_edge(START, "a")
builder.add_conditional_edges("a", route)   # after 'a', call route() to decide
builder.add_edge("b", "a")                   # 'b' loops back to 'a' -> a CYCLE
graph = builder.compile()
```

This is precisely the shape from Phase 1: node `a` is the model, node `b` is the tools, and `route` is the “did the model request a tool?” decision. Because `b` loops back to `a`, the agent can iterate.

Controlling loops: the recursion limit

A cyclic graph could in principle loop forever, so LangGraph enforces a **recursion limit** on the number of super-steps. Exceeding it raises `GraphRecursionError`, which you can catch. Always pair a loop with a real termination condition *and* a sane recursion limit as a backstop.

```
from langgraph.errors import GraphRecursionError

try:
    graph.invoke(inputs, {"recursion_limit": 25})
except GraphRecursionError:
    print("Agent hit the step limit without finishing.")
```

The Command API: update state AND route in one node

Sometimes a node needs to both update state and decide where to go next. Returning a `Command` object does both at once, removing the need for a separate conditional edge.

Command fuses a state update with a routing decision. The Literal annotation tells the graph the possible targets.

```
from langgraph.types import Command
from typing import Literal

def node_a(state: State) -> Command[Literal["node_b", "node_c"]]:
    return Command(
        update={"foo": "bar"},   # state update
        goto="node_b",           # control flow (where to go next)
    )
```

> **Command vs. conditional edges**
>
> Use a conditional edge when routing is a pure function of state and you want the decision visible as graph structure. Use Command when the routing decision is naturally produced alongside the state update inside the node (common in tools and multi-agent handoffs, covered in Phase 3).

4. ToolNode and bind_tools: the tool-calling loop

You met `bind_tools` in Phase 1: it tells the model which tools exist. In LangGraph, the other half of the loop — actually executing the requested tools and feeding results back — is handled by a **tool node**. You can write it yourself (instructive) or use the prebuilt `ToolNode` (production-ready).

Writing the tool node by hand

A hand-written tool node — the same logic as your Phase 1 loop, now as a graph node.

```
from langchain.messages import ToolMessage

tools_by_name = {t.name: t for t in tools}

def tool_node(state: MessagesState):
    """Execute every tool call the model just requested."""
    results = []
    for call in state["messages"][-1].tool_calls:
        tool = tools_by_name[call["name"]]
        observation = tool.invoke(call["args"])
        results.append(ToolMessage(content=str(observation), tool_call_id=call["id"]))
    return {"messages": results}
```

Using the prebuilt ToolNode

The prebuilt `ToolNode` does all of the above and more: it runs tool calls in parallel, handles tools that return `Command` objects, and manages errors. It is the recommended choice for real agents.

```
from langgraph.prebuilt import ToolNode

tool_node = ToolNode(tools)        # handles parallel calls, Command results, errors
builder.add_node("tools", tool_node)
```

> **The fastest path: create_agent**
> For the common case, LangChain's create_agent builds the entire ReAct graph for you — a model node, a ToolNode, and the conditional loop between them. We build the graph manually first (so you understand it), then show create_agent as the shortcut.

create_agent: the prebuilt ReAct agent. It IS a LangGraph graph under the hood.

```
from langchain.agents import create_agent

agent = create_agent(
    "anthropic:claude-sonnet-4-6",
    tools=[search, get_weather],
    system_prompt="You are a helpful assistant. Be concise and accurate.",
)
result = agent.invoke({"messages": [{"role": "user", "content": "Weather in SF?"}]})
```

5. Checkpointers: MemorySaver, SQLite, Postgres

So far our graphs are stateless between calls. A **checkpointer** changes that: when you compile a graph with one, LangGraph saves a snapshot of the state at every super-step, organised into **threads**. This single feature unlocks conversational memory, human-in-the-loop, time-travel debugging, and fault tolerance.

A **thread** is identified by a `thread_id` you pass in the config. Reusing the same id resumes the same conversation; a new id starts fresh. The thread is your persistent cursor into saved state.

A checkpointer + a thread_id give the graph memory across calls.

```
from langgraph.checkpoint.memory import InMemorySaver

checkpointer = InMemorySaver()
graph = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "user-123"}}
graph.invoke({"messages": [{"role": "user", "content": "Hi, I'm Ada."}]}, config)
# Later turn, SAME thread_id -> the graph still remembers the conversation:
graph.invoke({"messages": [{"role": "user", "content": "What's my name?"}]}, config)
```

Checkpointer	Package	Use for
`InMemorySaver`	built in (langgraph-checkpoint)	Experiments, tests, notebooks
`SqliteSaver`	langgraph-checkpoint-sqlite	Local apps, single-machine persistence
`PostgresSaver`	langgraph-checkpoint-postgres	Production, concurrent users

Swapping persistence backends is a one-line change — the rest of your graph is unchanged.

```
# SQLite (install: pip install langgraph-checkpoint-sqlite)
```

```
import sqlite3
from langgraph.checkpoint.sqlite import SqliteSaver
checkpointer = SqliteSaver(sqlite3.connect("agent.db", check_same_thread=False))

# Postgres (install: pip install langgraph-checkpoint-postgres)
from langgraph.checkpoint.postgres import PostgresSaver
checkpointer = PostgresSaver.from_conn_string("postgresql://user:pass@host/db")
checkpointer.setup()   # run once to create the tables
```

Inspecting and editing state

With a checkpointer you can read the current state, walk the full history, and even edit it:

- `graph.get_state(config)` returns a `StateSnapshot` with the current values, the next node, and metadata.
- `graph.get_state_history(config)` returns every checkpoint, newest first — this is your episodic memory and the basis for time travel.
- `graph.update_state(config, values)` writes a new checkpoint with edited values (passing through reducers), without destroying history.

> **Cross-thread memory: the Store**
>
> Checkpointers persist state *within* a thread. To share facts *across* threads (a user's preferences in every conversation) you compile the graph with a Store as well — the long-term memory from Phase 1. Pass store=... to compile() and read it via runtime.store inside nodes.

6. Human-in-the-loop with interrupt()

Agents often need a human to approve an action, edit a draft, or supply missing information before continuing. LangGraph's `interrupt()` function pauses the graph **at any point in a node**, saves state via the checkpointer, and waits — indefinitely — until you resume it with a value.

Three ingredients are required: a checkpointer (to save the paused state), a `thread_id` (so the runtime knows which state to resume), and a call to `interrupt()` where you want to pause.

A human approval gate. The graph parks itself until you resume the same thread with a decision.

```
from langgraph.types import interrupt, Command

def approval_node(state: State):
    decision = interrupt({                # pause; payload surfaces to the caller
        "question": "Approve this action?",
        "details": state["action_details"],
    })
    # On resume, Command(resume=...) becomes the return value of interrupt()
    return Command(goto="proceed" if decision else "cancel")

# 1) First run hits the interrupt and pauses:
config = {"configurable": {"thread_id": "approval-1"}}
result = graph.invoke({"action_details": "Transfer $500"}, config)
print(result["__interrupt__"])   # shows what the graph is waiting on

# 2) Resume with the human's answer:
graph.invoke(Command(resume=True), config)   # True -> proceed, False -> cancel
```

Common HITL patterns

- **Approve / reject:** pause before a critical action (payments, deletes, emails); resume with `True`/`False`.
- **Review and edit:** surface an LLM draft, let a human edit it, and resume with the corrected text, which becomes the interrupt's return value.
- **Interrupts inside tools:** put `interrupt()` inside a tool so the approval logic lives with the tool itself and is reusable.
- **Validate input:** loop on `interrupt()`, re-prompting until the human supplies valid data.

> **Three rules that will save you hours**
>
> (1) Never wrap interrupt() in a bare try/except — it works by raising a special exception, which you would swallow. (2) Keep interrupt() calls in the SAME order on every node execution; resume values are matched by index. (3) Side effects BEFORE an interrupt re-run when you resume, so make them idempotent or move them after the interrupt.

7. Streaming: values, updates, messages, custom

Agents can take many seconds across multiple steps. Streaming surfaces progress in real time, transforming the user experience. Call `graph.stream(...)` (or `astream`) with one or more **stream modes**. With `version="v2"` (LangGraph 1.1+) every chunk is a uniform `StreamPart` dict with `type`, `ns`, and `data` keys.

Stream mode	Yields	Best for
`values`	Full state after each step	Showing the complete evolving state
`updates`	Only the changed keys per node	Step-by-step progress / logging
`messages`	(token, metadata) from LLM calls	Token-by-token chat UIs
`custom`	Anything you emit via the writer	Tool progress bars, custom events

Streaming multiple modes at once. Filter on chunk['type'] to handle each.

```
for chunk in graph.stream(
    {"topic": "ice cream"},
    stream_mode=["updates", "messages"],
    version="v2",
):
    if chunk["type"] == "updates":
        for node, update in chunk["data"].items():
            print(f"[{node}] {update}")
    elif chunk["type"] == "messages":
        token, meta = chunk["data"]
        print(token.content, end="", flush=True)   # live token stream
```

> **Emitting custom progress**
>
> Inside a node or tool, call get_stream_writer() and write any JSON-serialisable payload (e.g. {'status': 'Retrieved 50/100 records'}). Subscribe with stream_mode='custom'. This is how you build progress indicators for long-running tools — even ones that wrap non-LangChain LLMs.

Project 1 — A ReAct agent from scratch (no prebuilt helpers)

PROJECT 1

Build the ReAct loop with raw StateGraph primitives

Goal: Recreate your Phase 1 agent the LangGraph way: a MessagesState, a model node, a hand-written tool node, and a conditional edge that loops. No create_agent, no ToolNode — so you see every wire.

```
import os
from typing import Literal
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langchain.messages import SystemMessage, ToolMessage, HumanMessage
from langgraph.graph import StateGraph, MessagesState, START, END

os.environ["ANTHROPIC_API_KEY"] = "sk-..."
llm = init_chat_model("claude-sonnet-4-6")

@tool
def add(a: float, b: float) -> float:
    """Add two numbers."""
    return a + b

@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

tools = [add, multiply]
tools_by_name = {t.name: t for t in tools}
llm_with_tools = llm.bind_tools(tools)

# --- Node 1: the model reasons over the conversation ---
def call_model(state: MessagesState):
    system = SystemMessage("You are a precise calculator. Use tools for all arithmetic.")
    response = llm_with_tools.invoke([system] + state["messages"])
    return {"messages": [response]}

# --- Node 2: execute any tool calls (hand-written tool node) ---
def tool_node(state: MessagesState):
    results = []
    for call in state["messages"][-1].tool_calls:
        observation = tools_by_name[call["name"]].invoke(call["args"])
        results.append(ToolMessage(content=str(observation), tool_call_id=call["id"]))
    return {"messages": results}

# --- The conditional edge: loop or finish? ---
def should_continue(state: MessagesState) -> Literal["tools", "__end__"]:
    if state["messages"][-1].tool_calls:
        return "tools"
    return END

# --- Wire the graph ---
builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
```

```
builder.add_node("tools", tool_node)
builder.add_edge(START, "call_model")
builder.add_conditional_edges("call_model", should_continue, ["tools", END])
builder.add_edge("tools", "call_model")        # the loop
agent = builder.compile()

out = agent.invoke({"messages": [HumanMessage("What is (12 + 8) times 3?")]})
for m in out["messages"]:
    m.pretty_print()
```

Run it and compare with Phase 1: the `while` loop became the `tools→call_model` edge, your “done?” check became `should_continue`, and the message list became `MessagesState`. Same skeleton, now durable and streamable.

> **Visualise it**
>
> Call agent.get_graph().draw_mermaid() to print the graph as Mermaid, or draw_mermaid_png() to render it. Seeing the call_model -> tools -> call_model cycle on screen makes the structure click.

Project 2 — A chatbot with persistent memory

PROJECT 2

A conversational agent that remembers across turns

Goal: Add a checkpointer so the same thread_id continues a conversation. The bot recalls earlier turns without you re-sending them, because state is checkpointed and reloaded per thread.

```
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.checkpoint.memory import InMemorySaver

llm = init_chat_model("claude-sonnet-4-6")

def chat(state: MessagesState):
    return {"messages": [llm.invoke(state["messages"])]}

builder = StateGraph(MessagesState)
builder.add_node("chat", chat)
builder.add_edge(START, "chat")
builder.add_edge("chat", END)

# The ONE line that adds memory:
checkpointer = InMemorySaver()       # swap for SqliteSaver/PostgresSaver in prod
bot = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "conv-1"}}

def say(text):
    out = bot.invoke({"messages": [{"role": "user", "content": text}]}, config)
    print("Bot:", out["messages"][-1].content)

say("Hi! My name is Ada and I love LangGraph.")
say("What's my name, and what do I love?")   # the bot remembers — same thread
# Start a NEW thread -> a clean slate:
config2 = {"configurable": {"thread_id": "conv-2"}}
out = bot.invoke({"messages": [{"role": "user", "content": "What's my name?"}]}, config2)
print("New thread:", out["messages"][-1].content)   # it does NOT know Ada here
```

Notice that you only ever send the *new* message each turn — never the whole history. The checkpointer reloads prior state for the thread automatically. Switch to `SqliteSaver` and the memory survives a process restart.

> **Going further: long-term memory**
>
> Per-thread memory forgets across conversations. To remember Ada's name in a brand-new thread, add a Store and write durable facts namespaced by user (Phase 1's external memory). Compile with store=... and read/write it via runtime.store in your node.

Project 3 — A tool-calling agent with a human approval step

PROJECT 3

Pause for human approval before a sensitive tool runs

Goal: Combine tools, a checkpointer, and interrupt() so the agent stops before sending an email, shows you the draft, and only sends it after you approve — the canonical HITL pattern.

```
import sqlite3
from langchain.chat_models import init_chat_model
from langchain.tools import tool
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.prebuilt import ToolNode
from langgraph.checkpoint.sqlite import SqliteSaver
from langgraph.types import interrupt, Command
from typing import Literal

@tool
def send_email(to: str, subject: str, body: str) -> str:
    """Send an email to a recipient."""
    # Pause for human approval BEFORE the side effect.
    decision = interrupt({
        "action": "send_email", "to": to, "subject": subject,
        "body": body, "message": "Approve sending this email?",
    })
    if decision.get("action") == "approve":
        # (optionally use edited fields from `decision`)
        return f"Email sent to {decision.get('to', to)}"
    return "Email cancelled by the user."

llm = init_chat_model("claude-sonnet-4-6").bind_tools([send_email])
tools_node = ToolNode([send_email])

def call_model(state: MessagesState):
    return {"messages": [llm.invoke(state["messages"])]}

def should_continue(state: MessagesState) -> Literal["tools", "__end__"]:
    return "tools" if state["messages"][-1].tool_calls else END

builder = StateGraph(MessagesState)
builder.add_node("call_model", call_model)
builder.add_node("tools", tools_node)
builder.add_edge(START, "call_model")
builder.add_conditional_edges("call_model", should_continue, ["tools", END])
builder.add_edge("tools", "call_model")

checkpointer = SqliteSaver(sqlite3.connect("approvals.db", check_same_thread=False))
agent = builder.compile(checkpointer=checkpointer)

config = {"configurable": {"thread_id": "email-1"}}
result = agent.invoke(
    {"messages": [{"role": "user",
                   "content": "Email alice@example.com to confirm Friday's 2pm meeting."}]},
    config,
)
```

```
print(result["__interrupt__"])      # the agent is paused, awaiting approval

# Human approves (optionally editing the subject):
final = agent.invoke(
    Command(resume={"action": "approve", "subject": "Confirming Friday 2pm"}),
    config,
)
print(final["messages"][-1].content)
```

The interrupt lives *inside the tool*, so the approval gate travels with `send_email` wherever it is used. Because the checkpointer is SQLite, you could approve hours later from a different process — the paused state is durable on disk.

> **Idempotency reminder**
> The tool re-runs from its start when you resume, so the real send must happen AFTER the interrupt() returns approval — never before it. This is the idempotency rule from section 6 in action.

> **Phase 2 checkpoint**
> You can now build a StateGraph with typed state and reducers, branch and loop with conditional edges and Command, run tools with a hand-written node or ToolNode, persist state with a checkpointer, pause for humans with interrupt(), and stream progress. You have built three working agents. Next, you compose them.

Coming in Phase 3 — Agentic Patterns: subgraphs, supervisor + worker multi-agent systems, map-reduce with the Send API, plan-and-execute, reflection / self-critique loops, agent handoffs, cross-graph communication, and LangGraph Studio for live visual debugging. Four projects build each pattern end to end.